Лабораторная работа №5 — Введение в Kubernetes

Цель

Развернуть приложение из ЛР №4 (FastAPI + PostgreSQL + Redis) в локальном кластере Kubernetes, используя стандартные ресурсы K8s вместо Docker Compose.

Инструменты

— **minikube** (или kind) — локальный Kubernetes-кластер
— **kubectl** — CLI для управления кластером
— Docker образ приложения, собранный в ЛР №4

Задание

1. Подготовка образа

Образ приложения должен быть доступен кластеру. Воспользуйтесь одним из вариантов:

— загрузить образ напрямую в minikube (`minikube image load`)
— либо опубликовать на Docker Hub и использовать `image: <username>/app:tag`

2. Развертывание базы данных и кэша

Создайте манифесты Kubernetes для **PostgreSQL** и **Redis**:

— `Deployment` для каждого сервиса
— `Service` (ClusterIP) для внутренней связи между подами
— Для PostgreSQL — `PersistentVolumeClaim` для хранения данных

3. Развертывание приложения

Создайте манифесты для FastAPI-приложения:

— `ConfigMap` — для не секретных переменных окружения (`APP_OWNER`, `REDIS_URL`, `CACHE_TTL` и т.д.)
— `Secret` — для чувствительных данных (`APP_SECRET`, `POSTGRES_PASSWORD`)

— `Deployment` — минимум 1 реплика; переменные окружения берутся из ConfigMap и Secret

— `Service` типа `NodePort` или с использованием `minikube service` — для доступа с хоста

4. Проверка

Убедитесь, что приложение работает корректно:

```
# Посмотреть все ресурсы в неймспейсе
kubectl get all

# Логи пода приложения
kubectl logs <pod-name>

# Открыть приложение
minikube service <service-name>
```

Проверьте те же эндпоинты, что и в ЛР №4:

— `GET /` — приветствие с `hostname` и `server_time`

— `GET /health` — статус PostgreSQL-соединения и Redis

— `GET /weather/{city}` — погода (первый запрос — `"source": "api"`, повторный — `"source": "cache"`)

— `POST /notes` + `GET /notes` — работа с заметками через PostgreSQL

Структура манифестов

Рекомендуется организовать файлы так:

```
k8s/
├── postgres-deployment.yaml
├── postgres-service.yaml
├── postgres-pvc.yaml
├── redis-deployment.yaml
├── redis-service.yaml
├── app-configmap.yaml
├── app-secret.yaml
├── app-deployment.yaml
└── app-service.yaml
```

Все манифесты применяются командой:

```
kubectl apply -f k8s/
```

Ключевые концепции для изучения

Концепция	Назначение
`Pod`	Минимальная единица запуска — один или несколько контейнеров
`Deployment`	Управляет репликами подов, обеспечивает обновление и восстановление
`Service`	Стабильный сетевой адрес для группы подов
`ConfigMap`	Хранение конфигурации (не секретной)
`Secret`	Хранение чувствительных данных (пароли, ключи)
`PersistentVolumeClaim`	Запрос на постоянное хранилище для базы данных

Требования к защите

1. Все поды в состоянии `Running` — скриншот `kubectl get pods`
2. Ответ `GET /health` с `"redis": true` и корректным статусом БД
3. Демонстрация кэширования: первый запрос `"source": "api"`, второй — `"source": "cache"`
4. В ответах API поле `hostname` содержит имя пода Kubernetes (не контейнера Docker)
5. `APP_OWNER` установлен в персональное имя студента (через ConfigMap)

Подсказки

— В Docker Compose сервисы обращаются друг к другу по имени сервиса (`db`, `redis`).

В Kubernetes роль имени сервиса играет `metadata.name` ресурса `Service`.

— Переменная `REDIS_URL` в приложении должна указывать на `Service` Redis: `redis://<redis-service-name>:6379/0`

— `DATABASE_URL` для PostgreSQL строится аналогично через имя `Service` PostgreSQL
— Если pod не запускается — `kubectl describe pod <pod-name>` и `kubectl logs <pod-name>` помогут найти причину